# WeatherWise.ai — Complete Architecture & Implementation Guide

Stateful Multi-Agent Weather Intelligence System

WeatherWise.ai Engineering

March 2026

## Contents

**1 Project Overview** **4**
1.1 Tech Stack . . . 4

**2 System Architecture** **5**
2.1 High-Level Graph Flow . . . 5
2.2 Two-Phase Execution Model . . . 5
2.3 Graph Wiring . . . 5

**3 State Management** **6**
3.1 WeatherState . . . 6
3.2 State Fields . . . 6
3.2.1 messages . . . 6
3.2.2 plan . . . 6
3.2.3 completed_indices . . . 6
3.2.4 fetched_weather . . . 6
3.2.5 retrieved_knowledge . . . 7
3.2.6 calculations . . . 7
3.2.7 routing . . . 7
3.2.8 intent . . . 7
3.3 Why Reducers Are Essential . . . 7
3.4 Worker Payloads . . . 8
3.4.1 FetchPayload (used by fetcher and retriever) . . . 8
3.4.2 CalcPayload (used by calculator) . . . 8

**4 Node Implementations** **9**
4.1 Supervisor Node . . . 9
4.1.1 Intent Buckets . . . 9
4.1.2 Structured Output . . . 9
4.1.3 Routing Logic . . . 9
4.1.4 Input Optimization . . . 10
4.1.5 Date Context . . . 10
4.1.6 Failure Handling . . . 10
4.2 Planner Node . . . 11

4.2.1 Structured Output . . . 11
4.2.2 Task Types . . . 11
4.2.3 Examples of Plan Decomposition . . . 11
4.2.4 Input . . . 12
4.2.5 State Updates . . . 12
4.2.6 Failure Handling . . . 12
4.3 Executor Node . . . 13
4.3.1 The Executor Node Function . . . 13
4.3.2 route_from_executor — The Routing Engine . . . 13
4.3.3 Send Payloads . . . 13
4.4 Fetcher Agent Node . . . 14
4.4.1 How It Works . . . 14
4.4.2 Available Xweather Tools . . . 14
4.4.3 State Updates . . . 14
4.4.4 Failure Handling . . . 14
4.4.5 Retry . . . 14
4.5 Retriever Agent Node . . . 15
4.5.1 How It Works . . . 15
4.5.2 RAG Architecture: Summary-to-Source Multi-Vector Retrieval . . . 15
4.5.3 State Updates . . . 15
4.5.4 Failure Handling . . . 15
4.6 Calculator Agent Node . . . 16
4.6.1 Available Tools . . . 16
4.6.2 Manual Tool-Calling Loop . . . 16
4.6.3 Input . . . 16
4.6.4 State Updates . . . 16
4.6.5 Failure Handling . . . 16
4.7 Synthesizer Node . . . 17
4.7.1 How It Works . . . 17
4.7.2 System Prompt Guidelines . . . 17
4.7.3 State Updates . . . 17
4.7.4 Failure Handling . . . 17

**5 Tools & External Integrations 18**
5.1 Xweather MCP Client . . . 18
5.1.1 Configuration . . . 18
5.1.2 Tool Caching . . . 18
5.2 ChromaDB Vector Search . . . 18
5.2.1 Runtime Retrieval (Lightweight) . . . 18
5.2.2 Configuration Constants . . . 18
5.2.3 Knowledge Base Corpus . . . 19
5.3 Math Operations . . . 19
5.3.1 compute_expression . . . 19
5.3.2 calculate_anomaly . . . 19
5.4 Offline Ingestion Pipeline . . . 19

**6 Infrastructure 20**
6.1 Retry Wrapper . . . 20

6.2 Structured Logging . . . 20
6.3 LangSmith Tracing . . . 20
6.4 Checkpointing . . . 20
6.4.1 Redis Checkpointer (Production) . . . 21
6.5 Prompt Caching . . . 21

**7 Server & Frontend** **23**
7.1 FastAPI Server . . . 23
7.1.1 Startup (Lifespan) . . . 23
7.1.2 REST Endpoint . . . 23
7.1.3 WebSocket Endpoint . . . 23
7.1.4 Streaming Implementation . . . 23
7.1.5 Static Frontend Serving . . . 24
7.2 React Frontend . . . 24
7.2.1 Pages . . . 24
7.2.2 WebSocket Communication . . . 24
7.2.3 Chat Types . . . 24
7.2.4 UI Components . . . 25
7.2.5 Intent Label Mapping . . . 25

**8 Complete Data Flow Example** **26**
8.1 Step 1: Supervisor . . . 26
8.2 Step 2: Planner . . . 26
8.3 Step 3: Executor (1st invocation) . . . 27
8.4 Step 4: Fetcher Agents (parallel) . . . 27
8.5 Step 5: Executor (2nd invocation) . . . 27
8.6 Step 6: Calculator Agent . . . 28
8.7 Step 7: Executor (3rd invocation) . . . 28
8.8 Step 8: Synthesizer . . . 28
8.9 Step 9: END . . . 29

**9 Project File Structure** **30**

**10 Environment Variables** **32**

**11 Design Principles** **33**

# 1 Project Overview

WeatherWise.ai is a stateful multi-agent weather intelligence system built on LangGraph. It handles compound natural-language queries that may require real-time weather lookups for multiple cities, vector database retrieval (RAG) for atmospheric science knowledge, and mathematical operations on weather data — all within a single conversational turn.

The system uses a **Planner-Executor pattern** with **dynamic parallel execution** (fan-out/fan-in). A planning LLM decomposes the user's query into independent sub-tasks, an executor fans them out to specialized worker agents in parallel via LangGraph's `Send` primitive, and a synthesizer weaves all gathered data into a final analyst-grade response.

## 1.1 Tech Stack

| Component | Technology |
|---|---|
| Orchestration | LangGraph v0.2+ (StateGraph with Send-based fan-out) |
| LLM Provider | Anthropic Claude (via `langchain-anthropic`) |
| Classifier Model | Claude Haiku 4.5 (fast, low-cost intent classification) |
| Planner/Synthesizer Model | Claude Sonnet 4 (higher capability for decomposition and prose) |
| Weather Data | Xweather MCP Server via `langchain-mcp-adapters` |
| RAG Vector Store | ChromaDB (local persistent) |
| Embedding Model | OpenAI `text-embedding-3-small` (1536 dimensions) |
| Document Parsing | `unstructured` library (`hi_res` strategy) — ingestion only |
| Math Operations | AST-based safe expression evaluator |
| Checkpointing | AsyncRedisSaver (production, 24h TTL), MemorySaver (dev), AsyncSqliteSaver (local persistent) |
| Prompt Caching | Anthropic native `cache_control` content blocks (GA, no beta header) |
| Validation | Pydantic models for structured LLM output |
| Logging | `structlog` (structured JSON or console output) |
| Retry | `tenacity` (exponential backoff for transient API failures) |
| Tracing | LangSmith (auto-enabled when `LANGCHAIN_API_KEY` is set) |
| Server | FastAPI with WebSocket streaming |
| Frontend | React + Vite + TailwindCSS + Framer Motion |
| Package Manager | `uv` (Python), npm (frontend) |
| Runtime | Python 3.11+ |

# 2 System Architecture

## 2.1 High-Level Graph Flow

The system processes each user query through a directed acyclic graph of seven nodes. The full execution path is:

```
START -> supervisor -> planner -> executor -> [workers] -> executor -> synthesizer -> END
```

The executor node is re-entrant — workers loop back to it after completing their tasks, and the executor decides whether to fan out more workers or proceed to synthesis. This enables two-phase execution where data-fetching tasks complete before calculation tasks that depend on that data.

## 2.2 Two-Phase Execution Model

The executor implements a two-phase strategy:

- **Phase 1 (Parallel):** All `fetch` (weather API) and `research` (RAG retrieval) tasks run simultaneously.
- **Phase 2 (Parallel, data-dependent):** All `calculate` (math) tasks run simultaneously, each receiving the complete set of weather data gathered in Phase 1.

This guarantees that calculators always have access to the fetched data they need, while still maximizing parallelism within each phase.

## 2.3 Graph Wiring

The graph is constructed in `src/weatherwise/graph.py` using LangGraph's `StateGraph`:

1. **START -> supervisor**: Every query begins with intent classification.
2. **supervisor -> planner OR END**: Conditional edge based on `routing` field. Weather intents proceed to the planner. Greetings and off-topic queries get a direct response and route to END.
3. **planner -> executor**: Unconditional edge.
4. **executor -> [workers] OR synthesizer**: Conditional edge using `route_from_executor`. Returns either a list of `Send` objects (fan-out to workers) or the string `"synthesizer"` (proceed to synthesis).
5. **fetcher_agent -> executor**: Workers loop back to executor.
6. **retriever_agent -> executor**: Workers loop back to executor.
7. **calculator_agent -> executor**: Workers loop back to executor.
8. **synthesizer -> END**: Unconditional edge.

The graph is compiled with a checkpointer (MemorySaver by default) and a recursion limit of 25 steps.

# 3 State Management

## 3.1 WeatherState

All nodes operate on a single shared `WeatherState` TypedDict defined in `src/weatherwise/state.py`. This is the central data structure that flows through the entire graph.

```
class WeatherState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]
    plan: list[dict[str, Any]]
    completed_indices: Annotated[list[int], operator.add]
    fetched_weather: Annotated[dict[str, Any], merge_dicts]
    retrieved_knowledge: Annotated[list[str], operator.add]
    calculations: Annotated[list[str], operator.add]
    routing: Literal["clear", "ambiguous"]
    intent: str
```

## 3.2 State Fields

### 3.2.1 messages

- **Type:** `list[BaseMessage]`
- **Reducer:** `add_messages` (LangChain built-in)
- **Written by:** Supervisor (for greeting/off-topic responses), Synthesizer (final response)
- **Purpose:** Maintains the full chat history. The `add_messages` reducer handles deduplication by message ID and auto-deserialization of message objects.

### 3.2.2 plan

- **Type:** `list[dict[str, Any]]`
- **Reducer:** None (last-write-wins overwrite semantics)
- **Written by:** Planner
- **Purpose:** The decomposed task list. Each dict has keys `op` (one of `"fetch"`, `"research"`, `"calculate"`), `location` (city name or empty string), and `query` (search/math query or empty string). Set once by the planner and never modified afterward.

### 3.2.3 completed_indices

- **Type:** `list[int]`
- **Reducer:** `operator.add` (list concatenation)
- **Written by:** All worker nodes (fetcher, retriever, calculator)
- **Purpose:** Tracks which plan task indices have been completed. Each worker appends its `plan_index` on completion. The executor checks this set to determine which tasks remain. The reducer ensures parallel workers safely append without overwriting each other.

### 3.2.4 fetched_weather

- **Type:** `dict[str, Any]`
- **Reducer:** `merge_dicts` (custom — merges two dicts, right values win on key conflicts)
- **Written by:** Fetcher agent

- **Purpose:** Weather API results keyed by lowercase location name. Each fetcher writes a different key (e.g., `{"boston": {...}}`). The `merge_dicts` reducer combines results from parallel fetchers into a single dict (e.g., `{"boston": {...}, "new york": {...}}`).

#### 3.2.5 retrieved_knowledge

- **Type:** `list[str]`
- **Reducer:** `operator.add` (list concatenation)
- **Written by:** Retriever agent
- **Purpose:** RAG retrieval results as formatted context strings with source citations. Parallel retrievers append their results, and the reducer concatenates them.

#### 3.2.6 calculations

- **Type:** `list[str]`
- **Reducer:** `operator.add` (list concatenation)
- **Written by:** Calculator agent
- **Purpose:** Math/calculation result strings. Parallel calculators append their results.

#### 3.2.7 routing

- **Type:** `Literal["clear", "ambiguous"]`
- **Reducer:** None (last-write-wins)
- **Written by:** Supervisor
- **Purpose:** Controls the conditional edge after the supervisor. `"clear"` routes to the planner (weather intent). `"ambiguous"` routes to END (greeting/off-topic with direct response).

#### 3.2.8 intent

- **Type:** `str`
- **Reducer:** None (last-write-wins)
- **Written by:** Supervisor
- **Purpose:** The classified intent label (e.g., `"real_time"`, `"math"`, `"greeting"`). Used by the WebSocket server to send an intent badge to the frontend.

### 3.3 Why Reducers Are Essential

When the executor fans out three fetcher workers in parallel, all three write to `fetched_weather` simultaneously. Without the `merge_dicts` reducer, LangGraph would use last-write-wins semantics, meaning the last worker to finish would overwrite the results from the other two. The reducer ensures all results are merged.

The same applies to `completed_indices` — three workers each returning `[0]`, `[1]`, and `[2]` get concatenated by `operator.add` into `[0, 1, 2]`, correctly tracking all completions.

Fields like `plan`, `routing`, and `intent` do not need reducers because they are written by a single node (planner or supervisor) and never modified by parallel workers.

## 3.4 Worker Payloads

Worker nodes do not receive the full `WeatherState`. They receive typed payloads via the `Send` primitive:

### 3.4.1 FetchPayload (used by fetcher and retriever)

```
class FetchPayload(TypedDict):
    plan_index: int   # Index in the plan list
    op: str           # "fetch" or "research"
    location: str     # City name
    query: str        # Search query
```

### 3.4.2 CalcPayload (used by calculator)

```
class CalcPayload(TypedDict):
    plan_index: int
    op: str                          # "calculate"
    location: str
    query: str
    fetched_weather: dict[str, Any] # Snapshot of all weather data from Phase 1
```

The executor constructs these payloads and passes them via `Send("node_name", payload)`. The key design detail is that `CalcPayload` includes the full `fetched_weather` dict, injected by the executor at routing time. This is how calculators access real weather data without needing to read from the shared state directly.

# 4 Node Implementations

## 4.1 Supervisor Node

**File:** `src/weatherwise/nodes/supervisor.py`

**Model:** Claude Haiku 4.5 (`claude-haiku-4-5-20251001`) — chosen for speed and low cost since intent classification is a lightweight task.

**Configuration:** `max_tokens=512`, `temperature=0`, `timeout=30s`

**Purpose:** Classifies the user's latest message into one of eight intent buckets and routes the query accordingly.

### 4.1.1 Intent Buckets

| Bucket | Description | Example |
|---|---|---|
| `real_time` | Direct weather lookups for a single location | "What is the temp in San Jose?" |
| `comparative` | Comparing weather between two or more locations | "Is it hotter in Phoenix or Miami?" |
| `lifestyle` | Activity/planning questions that depend on weather | "Is today good for a wedding in Napa?" |
| `safety` | Severe weather alerts, warnings, emergencies | "Any active tornado warnings?" |
| `explanation` | "Why" questions requiring atmospheric science | "Why is there a heat dome over the Midwest?" |
| `math` | Queries requiring data fetching AND calculations | "Add the temp of Boston and NYC in Kelvin" |
| `greeting` | Greetings, introductions, capability questions | "Hello", "What can you do?" |
| `off_topic` | Queries unrelated to weather/climate/outdoors | "What is the GDP of France?" |

### 4.1.2 Structured Output

The supervisor uses Pydantic structured output via `with_structured_output(RouteDecision)`:

```
class RouteDecision(BaseModel):
    bucket: Literal["real_time", "comparative", "lifestyle",
                    "safety", "explanation", "math",
                    "greeting", "off_topic"]
    reasoning: str
    response: str | None  # Only for greeting/off_topic
```

### 4.1.3 Routing Logic

- **Weather intents** (real_time, comparative, lifestyle, safety, explanation, math): Sets `routing="clear"` and `intent=<bucket>`. The graph proceeds to the planner.

- **Direct-response intents** (greeting, off_topic): The LLM generates a natural-language `response` in the structured output. The supervisor writes this as an `AIMessage` to state, sets `routing="ambiguous"`, and the graph routes to END — skipping the planner/executor/synthesizer pipeline entirely.

#### 4.1.4 Input Optimization

The supervisor receives only the last user message (`state["messages"][-1:]`), not the full history. This minimizes token usage for the classification call.

#### 4.1.5 Date Context

A date/time context string is prepended to the system prompt so the LLM knows the current date (important for weather relevance).

#### 4.1.6 Failure Handling

On any exception, the supervisor defaults to `routing="clear"` and `intent="real_time"`. This ensures the system never blocks — it degrades to treating the query as a basic weather lookup.

## 4.2 Planner Node

**File:** `src/weatherwise/nodes/planner.py`

**Model:** Claude Sonnet 4 (`claude-sonnet-4-20250514`) — chosen for higher reasoning capability needed to decompose complex compound queries.

**Configuration:** `max_tokens=500`, `temperature=0`, `timeout=60s`

**Purpose:** Decomposes the user query into a list of independent sub-tasks that can be executed in parallel (or in dependency order).

### 4.2.1 Structured Output

```python
class PlanTask(BaseModel):
    op: Literal["fetch", "research", "calculate"]
    location: str = ""
    query: str = ""

class PlanOutput(BaseModel):
    tasks: list[PlanTask]  # min_length=1
```

### 4.2.2 Task Types

| Op | Purpose | Worker |
|---|---|---|
| `fetch` | Get current weather data for a city via Xweather MCP | fetcher_agent |
| `research` | Search the ChromaDB knowledge base for background info | retriever_agent |
| `calculate` | Perform math on weather data (depends on fetch results) | calculator_agent |

### 4.2.3 Examples of Plan Decomposition

**Query:** "What's the weather in Tokyo?"

```
[{"op": "fetch", "location": "Tokyo", "query": ""}]
```

**Query:** "Compare weather in New York and London"

```
[
  {"op": "fetch", "location": "New York", "query": ""},
  {"op": "fetch", "location": "London", "query": ""}
]
```

**Query:** "Is Santos port at risk of closure due to wind? How does current wind compare to historical average?"

```
[
  {"op": "fetch", "location": "Santos", "query": ""},
  {"op": "research", "query": "Santos port wind closure thresholds", "location": ""},
  {"op": "calculate", "query": "wind anomaly Santos", "location": "Santos"}
]
```

#### 4.2.4 Input

The planner receives the full message history (`list(state["messages"])`), not just the last message. This allows it to understand conversational context.

#### 4.2.5 State Updates

Returns `{"plan": [...], "completed_indices": []}`. The `completed_indices` is reset to an empty list to start fresh tracking for the new plan.

#### 4.2.6 Failure Handling

On any exception, the planner creates a single fallback task: `[{"op": "fetch", "location": "<user_message>", "query": ""}]`. This ensures the system attempts at least a basic weather lookup rather than failing entirely.

## 4.3 Executor Node

**File:** `src/weatherwise/nodes/executor.py`

**Purpose:** Orchestrates the two-phase fan-out/fan-in pattern. The executor node itself is stateless — it returns an empty dict. All routing logic lives in the `route_from_executor` function, which serves as a conditional edge router.

### 4.3.1 The Executor Node Function

```python
async def executor(state: WeatherState) -> dict[str, Any]:
    return {}
```

The executor exists as a node only so that workers can loop back to it. The actual decision-making happens in the conditional edge.

### 4.3.2 route_from_executor — The Routing Engine

This function examines the plan and completed indices to determine what to do next:

1. **Safety check:** If `len(completed_indices) >= MAX_EXECUTOR_ITERATIONS` (10), force-route to `"synthesizer"` to prevent infinite loops.
2. **Categorize remaining tasks:** Iterate over the plan. For each uncompleted task:
   - `fetch` or `research` -> add to `phase1_sends` list
   - `calculate` -> add to `phase2_sends` list
   - Unknown op -> skip and log a warning
3. **Phase 1:** If `phase1_sends` is non-empty, return the list of `Send` objects. LangGraph spawns these workers in parallel.
4. **Phase 2:** If `phase1_sends` is empty but `phase2_sends` is non-empty, all fetch/research is done. Return `Send` objects for calculators. Each calculator's payload includes the full `fetched_weather` dict from state.
5. **Done:** If both lists are empty, return `"synthesizer"` to proceed to final synthesis.

### 4.3.3 Send Payloads

The executor constructs typed payloads for each worker:

- **Fetchers:** `Send("fetcher_agent", {plan_index, op, location, query})`
- **Retrievers:** `Send("retriever_agent", {plan_index, op, location, query})`
- **Calculators:** `Send("calculator_agent", {plan_index, op, location, query, fetched_weather})`

The critical detail for calculators is that `fetched_weather` is read from state at routing time and injected into the payload. This is the mechanism that passes Phase 1 results into Phase 2 workers.

## 4.4 Fetcher Agent Node

**File:** `src/weatherwise/nodes/fetcher.py`

**Model:** Claude Haiku 4.5 (`claude-haiku-4-5-20251001`) as a ReAct agent.

**Configuration:** `max_tokens=1024`, `temperature=0`, `timeout=45s`

**Purpose:** Retrieves real-time weather data for a specific location by invoking tools from the Xweather MCP server.

### 4.4.1 How It Works

1. **Loads MCP tools:** Calls `get_xweather_tools()` to get the list of available Xweather tools (cached with 5-minute TTL).
2. **Constructs prompt:** Either the task's query + location, or a default "Get current weather conditions for {location}".
3. **Runs ReAct agent:** Creates a LangChain agent using `create_agent(llm, tools)` with the Haiku model. The agent autonomously decides which Xweather tool to call based on the prompt.
4. **Extracts result:** Pulls the final AI message content from the agent's message history.

### 4.4.2 Available Xweather Tools

The system prompt instructs the agent to choose from:

- `xweather_get_current_weather` — current conditions
- `xweather_get_forecast_weather` — forecasts
- `xweather_get_current_airquality` — air quality
- `xweather_get_current_active_weather_alerts` — alerts and warnings

### 4.4.3 State Updates

```
{
    "fetched_weather": {location.lower().strip(): {
        "location": location,
        "raw_response": raw_response,
    }},
    "completed_indices": [plan_index],
}
```

The location key is normalized to lowercase for consistent merging.

### 4.4.4 Failure Handling

On any exception, returns an error message as the `raw_response` value instead of crashing. The synthesizer will see this error and report it honestly.

### 4.4.5 Retry

The `_invoke_fetch_agent` function is decorated with `@llm_retry(max_attempts=3)`, providing exponential backoff on rate limits, timeouts, and connection errors.

## 4.5 Retriever Agent Node

**File:** `src/weatherwise/nodes/retriever.py`

**Purpose:** Searches the ChromaDB knowledge base for atmospheric science context relevant to the user's query. Unlike the fetcher, this node does not use an LLM — it performs a direct vector similarity search.

### 4.5.1 How It Works

1. Calls `search_knowledge_base(query)` from `src/weatherwise/tools/vector_search.py`.
2. The function queries ChromaDB using OpenAI `text-embedding-3-small` embeddings.
3. Results are filtered by cosine distance threshold (1.2), limited to top-8 candidates, then top-5 results.
4. For each match, the parent document is loaded from the file-based docstore (`data/docstore/`) to get the full original content.
5. Results are formatted with source citations including PDF name, page number, and element type (text, table, chart/figure).

### 4.5.2 RAG Architecture: Summary-to-Source Multi-Vector Retrieval

The knowledge base uses a two-tier storage strategy:

- **ChromaDB** stores summary embeddings — concise LLM-generated summaries of each document chunk.
- **File-based docstore** (`data/docstore/`) stores the original parent documents — full text, table data, or image analysis.

At query time, ChromaDB returns the most relevant summaries, then the retriever fetches the corresponding full parent documents from the docstore. This means the LLM gets complete context, not just short summaries.

### 4.5.3 State Updates

```
{
    "retrieved_knowledge": [formatted_string_1, formatted_string_2, ...],
    "completed_indices": [plan_index],
}
```

### 4.5.4 Failure Handling

On any exception, returns a single-element list with the error message. If the knowledge base directory doesn't exist or is empty, returns an informative message rather than crashing.

## 4.6 Calculator Agent Node

**File:** `src/weatherwise/nodes/calculator.py`

**Model:** Claude Haiku 4.5 (`claude-haiku-4-5-20251001`) with tool-calling.

**Configuration:** `max_tokens=512`, `temperature=0`, `timeout=30s`

**Purpose:** Performs mathematical operations on fetched weather data. The LLM decides which math tools to call based on the data and the calculation request.

### 4.6.1 Available Tools

1. **`compute_expression`** — Evaluates arithmetic expressions safely using AST-based parsing. Supports `+`, `-`, `*`, `/` and unary operators. No `eval()` — the expression is parsed into an AST and evaluated node by node, preventing code injection.
2. **`calculate_anomaly`** — Compares a current weather value against a historical baseline average. Has built-in historical averages for select cities (Santos, New York, London, Tokyo, Mumbai) for `temp_c`, `wind_kph`, and `humidity_pct`.

### 4.6.2 Manual Tool-Calling Loop

The calculator implements a manual ReAct-style loop (not using `create_agent`):

```
for up to 5 iterations:
    1. Send messages to LLM (with tools bound)
    2. If LLM returns tool_calls -> execute each tool, append ToolMessage
    3. If LLM returns text (no tool_calls) -> that's the final answer, break
```

The loop is bounded at 5 iterations to prevent runaway execution.

### 4.6.3 Input

Receives a `CalcPayload` which includes the full `fetched_weather` dict collected during Phase 1. The calculator formats all weather data into a text summary and passes it to the LLM along with the calculation request.

### 4.6.4 State Updates

```
{
    "calculations": [result_string],
    "completed_indices": [plan_index],
}
```

### 4.6.5 Failure Handling

On any exception, returns an error message as the calculation result string.

## 4.7 Synthesizer Node

**File:** `src/weatherwise/nodes/synthesizer.py`

**Model:** Claude Sonnet 4 (`claude-sonnet-4-20250514`) — chosen for its ability to produce well-structured, insightful prose.

**Configuration:** `max_tokens=1500`, `temperature=0.3` (slightly creative for natural-sounding output), `timeout=60s`

**Purpose:** Reads all gathered data from state and composes the final analyst-grade response.

### 4.7.1 How It Works

1. **Builds a data block** from three state fields:
   - `fetched_weather` — formatted as "WEATHER DATA:" section
   - `retrieved_knowledge` — formatted as "RESEARCH CONTEXT:" section
   - `calculations` — formatted as "CALCULATIONS:" section
2. **Extracts the original user query** from the message history (finds the last `HumanMessage`).
3. **Invokes the LLM** with a system prompt instructing it to synthesize everything into a clear, markdown-formatted response with specific numbers, units, and source citations.
4. **Tags the invocation** with `config={"tags": ["synthesizer"]}`. This tag is critical for the WebSocket server — it uses this tag to filter `on_chat_model_stream` events and only stream tokens from the synthesizer, not from intermediate LLM calls in other nodes.

### 4.7.2 System Prompt Guidelines

- Lead with the direct answer
- Use markdown formatting (bold for key values, headers for sections)
- Include specific numbers with units
- Weave in the "why" from research context
- Explain calculation results
- Be honest about missing data

### 4.7.3 State Updates

```
{"messages": [AIMessage(content=final_response)]}
```

The synthesizer is the only node (besides supervisor for direct responses) that writes to `messages`. This final `AIMessage` is what the user sees.

### 4.7.4 Failure Handling

On exception, returns the raw data block with an error note, so the user still sees the gathered data even if synthesis fails.

# 5 Tools & External Integrations

## 5.1 Xweather MCP Client

**File:** `src/weatherwise/tools/xweather_mcp.py`

The Xweather MCP (Model Context Protocol) server provides real-time weather data. The client connects via `langchain-mcp-adapters` using the `streamable_http` transport.

### 5.1.1 Configuration

- **URL:** `https://mcp.api.xweather.com/mcp?api_key=<client_id>_<client_secret>`
- **Credentials:** `XWEATHER_CLIENT_ID` and `XWEATHER_CLIENT_SECRET` from environment variables.

### 5.1.2 Tool Caching

MCP tool schema negotiation adds 2-3 seconds of latency per round-trip. To avoid this on every fetcher call (especially during fan-out where multiple fetchers start simultaneously), tools are cached at the module level:

- **Cache TTL:** 300 seconds (5 minutes)
- **Concurrency safety:** An `asyncio.Lock` prevents thundering herd — when three parallel fetchers all start at once, only the first actually negotiates with the MCP server; the others wait on the lock and then get the cached result.
- **Cache invalidation:** `invalidate_tool_cache()` function available for testing.

## 5.2 ChromaDB Vector Search

**File:** `src/weatherwise/tools/vector_search.py`

### 5.2.1 Runtime Retrieval (Lightweight)

This module handles only the query-time retrieval path:

1. Initializes ChromaDB client pointing to `data/chroma_db/` directory.
2. Uses OpenAI `text-embedding-3-small` for query embedding.
3. Performs similarity search with score, filters by cosine distance threshold.
4. Loads parent documents from `data/docstore/` (JSON files keyed by `doc_id`).

### 5.2.2 Configuration Constants

| Constant | Value | Purpose |
|---|---|---|
| `RELEVANCE_THRESHOLD` | 1.2 | Maximum cosine distance for results |
| `TOP_K_CANDIDATES` | 8 | Initial candidates from ChromaDB |
| `TOP_K_RESULTS` | 5 | Final results after filtering |
| `COLLECTION_NAME` | `weatherwise_knowledge` | ChromaDB collection name |

#### 5.2.3 Knowledge Base Corpus

As of the latest ingestion, the knowledge base contains 429 vectors across these documents:

- CRS Extreme Heat report (54 vectors)
- Frontiers Heatwave paper (42 vectors)
- GOES satellite imagery (1 vector)
- NOAA Tornado December 2021 report (40 vectors)
- NOAA Climate Risk CPO003 report (156 vectors)
- NCA5 Chapter 2 (91 vectors)
- AMS/NASA Great Lakes Snowfall study (42 vectors)
- Satellite images — wildfire smoke, Helene VIIRS, Helene PACE (3 vectors)

### 5.3 Math Operations

**File:** `src/weatherwise/tools/math_ops.py`

#### 5.3.1 compute_expression

A safe arithmetic evaluator that uses Python's `ast` module to parse expressions into an Abstract Syntax Tree. Only numeric constants and basic operators (`+`, `-`, `*`, `/`, unary `-`, unary `+`) are supported. This prevents code injection attacks — no `eval()` is used.

#### 5.3.2 calculate_anomaly

Compares a current weather value against hardcoded historical baselines. Returns a human-readable anomaly string (e.g., "Boston temp_c anomaly: 28.5 vs historical avg 10.5 = +18.00 (18.0 above normal)").

Both functions are decorated with `@tool` from `langchain_core.tools`, making them available for LLM tool-calling.

### 5.4 Offline Ingestion Pipeline

**File:** `scripts/build_knowledge.py` (not imported at runtime)

The ingestion pipeline processes PDF documents into the ChromaDB knowledge base:

1. **Parse** PDFs using `unstructured` with `hi_res` strategy — extracts text chunks, tables, and images.
2. **Summarize** each element using Claude (via Anthropic API) — generates concise summaries for embedding.
3. **Resize** images to stay under Claude Vision's 5MB limit using Pillow (iterative quality/dimension reduction).
4. **Embed** summaries using OpenAI `text-embedding-3-small`.
5. **Store** summary vectors in ChromaDB and full parent documents in the file-based docstore.

This pipeline is intentionally separated from the runtime code per the project's RAG Separation principle: heavy ingestion dependencies (`unstructured`, PDF parsing, image processing) are never imported at runtime.

# 6 Infrastructure

## 6.1 Retry Wrapper

**File:** `src/weatherwise/retry.py`

All LLM and API calls are wrapped with a retry decorator based on `tenacity`:

- **Retryable exceptions:** `RateLimitError`, `APITimeoutError`, `APIConnectionError`, `TimeoutError`, `ConnectionError`, `OSError`
- **Strategy:** Exponential backoff — multiplier=1, min=1s, max=30s
- **Max attempts:** 3 (configurable per call site)
- **Logging:** Logs a warning before each retry sleep with the attempt number and exception.

## 6.2 Structured Logging

**File:** `src/weatherwise/logging.py`

Uses `structlog` for structured logging with two output modes:

- **Console mode** (default, development): Human-readable colored output via `ConsoleRenderer`.
- **JSON mode** (`LOG_FORMAT=json`): Machine-parseable JSON output via `JSONRenderer`.

Every node logs structured events at key points: `supervisor_start`, `supervisor_result`, `planner_start`, `planner_result`, `fetcher_start`, `fetcher_success`, `fetcher_failed`, etc. This enables easy debugging and monitoring.

Noisy libraries (`httpx`, `httpcore`, `chromadb`) are set to WARNING level.

## 6.3 LangSmith Tracing

**File:** `src/weatherwise/tracing.py`

When `LANGCHAIN_API_KEY` is set in the environment, LangSmith tracing is automatically enabled. All LangGraph nodes, LLM calls, tool invocations, and state transitions are traced and visible in the LangSmith dashboard.

Configuration:

- `LANGCHAIN_TRACING_V2`: Auto-set to `"true"` when API key is present.
- `LANGCHAIN_PROJECT`: Defaults to `"weatherwise"`.

The `get_run_config()` helper builds LangGraph-compatible configs with custom tags and metadata for trace filtering.

## 6.4 Checkpointing

The graph supports four checkpointer modes, selected via the `CHECKPOINTER_MODE` environment variable (or the `--checkpointer` CLI flag):

| Mode | Backend | Use Case |
|---|---|---|
| `memory` | `MemorySaver` (in-memory) | Development and testing (default) |
| `sqlite` | `AsyncSqliteSaver` | Local persistent storage |

| Mode | Backend | Use Case |
|---|---|---|
| `redis` | `AsyncRedisSaver` | Production (multi-node, persistent across restarts) |
| `none` | No checkpointer | Stateless operation |

Checkpointing enables conversation persistence — the graph can be resumed with the same `thread_id` and it will have access to the full state from previous turns.

### 6.4.1 Redis Checkpointer (Production)

The Redis checkpointer uses `AsyncRedisSaver` from `langgraph-checkpoint-redis`. Because `AsyncRedisSaver.__init__` calls `asyncio.get_running_loop()`, it must be created inside an async context — not at module level.

**In `server.py`:** A FastAPI lifespan async context manager creates the `AsyncRedisSaver` via `from_conn_string()` (which handles `asetup()` to create Redis search indices) and passes it to `build_graph(checkpointer_instance=saver)`.

**In `main.py`:** The `run_interactive()` async function wraps graph creation inside an `async with AsyncRedisSaver.from_conn_string(...)` block.

**`build_graph()` accepts a `checkpointer_instance` parameter:** When provided (e.g., an `AsyncRedisSaver`), it takes precedence over the `checkpointer` string. This keeps graph wiring logic sync while allowing callers to inject async checkpointers created in their own async contexts.

**TTL Configuration:**

- `default_ttl: 1440` — checkpoint keys expire after 24 hours (1440 minutes).
- `refresh_on_read: True` — accessing a session resets its TTL, so active conversations never expire mid-use.

**Environment:** `REDIS_URL` (default: `redis://localhost:6379`).

## 6.5 Prompt Caching

Anthropic prompt caching reduces latency and cost by caching the prefix of the message sequence on Anthropic's servers. Cached prefixes are served at up to 90% reduced cost for 5 minutes after first use.

**Implementation:** Messages are structured as content block lists with `cache_control` markers:

```
SystemMessage(content=[
    {"type": "text", "text": SYSTEM_PROMPT,
     "cache_control": {"type": "ephemeral"}}
])
```

No beta header is needed — prompt caching is GA with Anthropic SDK v0.77.0 and `langchain-anthropic` v1.3.1.

**Nodes with prompt caching enabled:**

| Node | What's cached | Model | Rationale |
|---|---|---|---|
| Synthesizer | System prompt + data block (both as `cache_control` breakpoints) | Sonnet 4 | Most expensive model; system prompt is static, data block is stable within a session |
| Planner | System prompt (with task decomposition examples) | Sonnet 4 | ~400 token system prompt, identical on every call |
| Supervisor | System prompt (with date context prepended) | Haiku 4.5 | ~1,200 tokens, called on every user message; date context changes per-minute but cache TTL is 5 min, so bursts hit |

**Cache consistency optimization:** The synthesizer sorts `fetched_weather` dictionary keys alphabetically when building the data block. This ensures that identical data sets produce identical prompt prefixes regardless of the order parallel fetchers completed, maximizing cache hit rate.

# 7 Server & Frontend

## 7.1 FastAPI Server

**File:** `src/weatherwise/server.py`

### 7.1.1 Startup (Lifespan)

The server uses a FastAPI lifespan async context manager to initialize the graph. When `CHECKPOINTER_MODE=redis`, the lifespan creates an `AsyncRedisSaver` via `from_conn_string()` and passes it to `build_graph(checkpointer_instance=saver)`. For other modes, the graph is built with the standard sync checkpointers. The graph instance is stored in a module-level `_graph` variable that all endpoints reference.

### 7.1.2 REST Endpoint

`POST /api/query` — Synchronous (non-streaming) query endpoint. Accepts `{"query": "...", "thread_id": "..."}`, runs `_graph.ainvoke()`, and returns `{"response": "...", "thread_id": "..."}`.

`GET /api/health` — Health check, returns `{"status": "ok"}`.

### 7.1.3 WebSocket Endpoint

`WS /ws/chat` — Bidirectional streaming chat. This is the primary interface used by the frontend.

**Client sends:**

```
{"type": "user_message", "content": "What's the weather in SF?"}
```

**Server sends (in order during processing):**

| Message Type | Fields | When Sent |
|---|---|---|
| `connected` | `session_id` | On WebSocket connection |
| `intent` | `intent`, `label` | After supervisor classifies the query |
| `tool_call` | `name`, `args`, `call_id` | When any tool starts executing |
| `tool_result` | `name`, `content`, `call_id` | When any tool finishes executing |
| `token` | `content` | Each token from the synthesizer's streaming output |
| `done` | `full_content` | When the full response is complete |
| `error` | `message` | On any error during processing |

### 7.1.4 Streaming Implementation

The server uses `_graph.astream_events(version="v2")` to receive fine-grained events from the graph:

- **Intent detection:** Listens for `on_chain_end` events from the `"supervisor"` node and extracts the `intent` field from the output.
- **Tool tracking:** Listens for `on_tool_start` and `on_tool_end` events to relay tool activity to the frontend.
- **Token streaming:** Listens for `on_chat_model_stream` events, but only forwards tokens tagged with `"synthesizer"`. This prevents intermediate LLM outputs (from fetcher/calculator agents) from leaking to the user.
- **Fallback:** If streaming captured no tokens (e.g., the synthesizer completed without streaming), the server reads the final state via `_graph.aget_state()` and extracts the last AI message.

#### 7.1.5 Static Frontend Serving

When `SERVE_FRONTEND=1` is set, the server mounts the frontend build directory (`frontend/dist/`) as static files at `/`.

### 7.2 React Frontend

**Directory:** `frontend/`

**Stack:** React, Vite, TailwindCSS, Framer Motion, React Router, tsparticles

#### 7.2.1 Pages

- **Landing Page** (`/`): Hero section with animated weather particle background, capability grid showcasing the system's six weather intent types.
- **Chat Page** (`/chat`): Full chat interface with message bubbles, typing indicator, tool call visualization, intent badges, and auto-scroll.

#### 7.2.2 WebSocket Communication

The `useWeatherChat` hook (`frontend/src/hooks/useWeatherChat.ts`) manages the WebSocket connection:

- Auto-connects on mount to `WS_URL` (configurable via `VITE_WS_URL` env var, defaults to `ws://<host>/ws/chat`).
- Auto-reconnects after 3 seconds on disconnect.
- Handles all server message types: `connected`, `intent`, `tool_call`, `tool_result`, `token`, `done`, `error`.
- Maintains message state with streaming support — assistant messages start with `isStreaming: true` and tokens are appended incrementally.

#### 7.2.3 Chat Types

```
interface ChatMessage {
  id: string;
  role: "user" | "assistant" | "system";
  content: string;
  intent?: string;
  intentLabel?: string;
  toolCalls?: ToolCall[];
```

```
  isStreaming?: boolean;
  timestamp: Date;
}

interface ToolCall {
  callId: string;
  name: string;
  args: Record<string, unknown>;
  result?: string;
  status: "running" | "complete" | "error";
}
```

#### 7.2.4 UI Components

| Component | Purpose |
|---|---|
| `ChatInput` | Text input with send button |
| `MessageBubble` | Renders user/assistant messages with markdown |
| `MessageList` | Scrollable message container |
| `ToolCallStep` | Shows tool name, args, and result in a collapsible card |
| `IntentBadge` | Colored badge showing the classified intent |
| `TypingIndicator` | Animated dots during streaming |
| `WelcomeMessage` | Initial greeting shown before first message |
| `ApprovalPrompt` | UI for human-in-the-loop approval (wired but not yet triggered by backend) |
| `Navbar` | Top navigation bar |
| `HeroSection` | Landing page hero with CTA |
| `CapabilityCard` / `CapabilityGrid` | Landing page feature showcase |
| `ParticleBackground` / `WeatherElements` | Animated weather particle effects |
| `GlassCard` / `GradientText` | Reusable styled components |

#### 7.2.5 Intent Label Mapping

The frontend maps intent strings to display labels:

```
{
  real_time: "Real-Time Forecast",
  comparative: "Location Comparison",
  lifestyle: "Lifestyle Advisory",
  safety: "Safety & Alerts",
  explanation: "Atmospheric Science",
  math: "Weather Math",
}
```

# 8 Complete Data Flow Example

This section traces a compound query through the entire system to show exactly how data moves between nodes.

**User Query:** *"Add the temperature of Boston and New York and give the answer in Kelvin"*

## 8.1 Step 1: Supervisor

**Input:** The user's `HumanMessage`.

**LLM call:** Haiku 4.5 with structured output.

**Decision:**

```
{
  "bucket": "math",
  "reasoning": "Query requires fetching weather data AND performing calculations",
  "response": null
}
```

**State updates:**

```
{"routing": "clear", "intent": "math"}
```

**Routing:** `route_from_supervisor` sees `routing="clear"`, routes to `"planner"`.

## 8.2 Step 2: Planner

**Input:** Full message history.

**LLM call:** Sonnet 4 with structured output.

**Plan generated:**

```
[
  {"op": "fetch", "location": "Boston", "query": ""},
  {"op": "fetch", "location": "New York", "query": ""},
  {"op": "calculate", "location": "", "query": "add temperatures of Boston and New York, conve
]
```

**State updates:**

```
{
  "plan": [
    {"op": "fetch", "location": "Boston", "query": ""},
    {"op": "fetch", "location": "New York", "query": ""},
    {"op": "calculate", "location": "", "query": "add temperatures..."}
  ],
  "completed_indices": []
}
```

### 8.3 Step 3: Executor (1st invocation)

**State check:** `completed_indices = []`, plan has 3 tasks.

**Categorization:**

- Index 0: `fetch` -> Phase 1
- Index 1: `fetch` -> Phase 1
- Index 2: `calculate` -> Phase 2

**Decision:** Phase 1 has tasks, return:

```
[
  Send("fetcher_agent", {"plan_index": 0, "op": "fetch", "location": "Boston", "query": ""}),
  Send("fetcher_agent", {"plan_index": 1, "op": "fetch", "location": "New York", "query": ""})
]
```

LangGraph spawns both fetcher instances in parallel.

### 8.4 Step 4: Fetcher Agents (parallel)

**Fetcher for Boston:**

1. Loads cached Xweather MCP tools.
2. Constructs prompt: "Get current weather conditions for Boston".
3. ReAct agent calls `xweather_get_current_weather` with location "Boston".
4. Extracts raw weather data from agent response.

**Returns:**

```
{
  "fetched_weather": {"boston": {"location": "Boston", "raw_response": "Current conditions: 72
  "completed_indices": [0]
}
```

**Fetcher for New York** (running simultaneously):

**Returns:**

```
{
  "fetched_weather": {"new york": {"location": "New York", "raw_response": "Current conditions
  "completed_indices": [1]
}
```

**Reducer merging (automatic by LangGraph):**

- `fetched_weather`: `merge_dicts` combines -> `{"boston": {...}, "new york": {...}}`
- `completed_indices`: `operator.add` concatenates -> `[0, 1]`

Both fetchers' edges route back to `executor`.

### 8.5 Step 5: Executor (2nd invocation)

**State check:** `completed_indices = [0, 1]`, completed set = `{0, 1}`.

**Categorization:**

- Index 0: completed, skip
- Index 1: completed, skip
- Index 2: `calculate` -> Phase 2

**Decision:** Phase 1 is empty, Phase 2 has tasks. Return:

```
[
  Send("calculator_agent", {
    "plan_index": 2,
    "op": "calculate",
    "location": "",
    "query": "add temperatures...",
    "fetched_weather": {"boston": {...}, "new york": {...}}
  })
]
```

Note: the executor injects the current `fetched_weather` from state into the calculator's payload.

### 8.6 Step 6: Calculator Agent

1. Formats all weather data into a text summary.
2. Sends to Haiku 4.5 with math tools bound.
3. LLM extracts temperatures (72°F and 68°F) from the data.
4. LLM calls `compute_expression` with `"(72 + 68 - 64) * 5/9 + 273.15"` (or equivalent conversion logic).
5. Tool returns the result.
6. LLM produces final answer text.

**Returns:**

```
{
  "calculations": ["The sum of Boston (72°F) and New York (68°F) temperatures converted to Kel
  "completed_indices": [2]
}
```

**State after reducer merging:** `completed_indices = [0, 1, 2]`

### 8.7 Step 7: Executor (3rd invocation)

**State check:** All 3 tasks completed. Phase 1 empty, Phase 2 empty.

**Decision:** Return `"synthesizer"`.

### 8.8 Step 8: Synthesizer

**Reads from state:**

- `fetched_weather`: `{"boston": {...}, "new york": {...}}`
- `retrieved_knowledge`: `[]` (no research tasks in this query)
- `calculations`: `["The sum of Boston (72°F)..."]`

**Builds data block:**

```
WEATHER DATA:
  boston: {'location': 'Boston', 'raw_response': 'Current conditions: 72°F...'}
  new york: {'location': 'New York', 'raw_response': 'Current conditions: 68°F...'}

CALCULATIONS:
  - The sum of Boston (72°F) and New York (68°F) temperatures...
```

**LLM call:** Sonnet 4, temperature=0.3, tagged with `["synthesizer"]`.

**Produces:** A well-formatted markdown response explaining the temperatures, the conversion process, and the final Kelvin result.

**State updates:**

```
{"messages": [AIMessage(content="## Temperature Sum in Kelvin\n\n**Boston** is currently...")]
```

### 8.9 Step 9: END

The graph terminates. The final `AIMessage` is delivered to the user via the WebSocket (streamed token-by-token during synthesis) or the REST API (returned as the full response).

# 9 Project File Structure

```
WeatherWise.ai/
|-- src/
|   |-- weatherwise/
|   |   |-- __init__.py
|   |   |-- state.py          # WeatherState, FetchPayload, CalcPayload
|   |   |-- graph.py          # StateGraph builder and wiring
|   |   |-- main.py           # CLI entry point
|   |   |-- server.py         # FastAPI + WebSocket server
|   |   |-- logging.py        # structlog configuration
|   |   |-- retry.py          # tenacity retry decorator
|   |   |-- tracing.py        # LangSmith tracing setup
|   |   |-- nodes/
|   |   |   |-- __init__.py
|   |   |   |-- supervisor.py # Intent classification + routing
|   |   |   |-- planner.py    # Query decomposition
|   |   |   |-- executor.py   # Two-phase fan-out orchestration
|   |   |   |-- fetcher.py    # Xweather MCP weather data retrieval
|   |   |   |-- retriever.py  # ChromaDB RAG search
|   |   |   |-- calculator.py # Math operations with tool-calling
|   |   |   |-- synthesizer.py# Final response composition
|   |   |-- tools/
|   |       |-- __init__.py
|   |       |-- xweather_mcp.py   # MCP client config + tool caching
|   |       |-- vector_search.py  # ChromaDB retrieval (runtime)
|   |       |-- math_ops.py       # Safe math evaluator + anomaly calc
|-- scripts/
|   |-- build_knowledge.py    # Offline RAG ingestion pipeline
|   |-- verify_retrieval.py   # RAG retrieval test harness
|-- frontend/
|   |-- src/
|   |   |-- App.tsx            # Router setup
|   |   |-- main.tsx           # React entry point
|   |   |-- pages/
|   |   |   |-- LandingPage.tsx
|   |   |   |-- ChatPage.tsx
|   |   |-- components/
|   |   |   |-- chat/          # Chat UI components
|   |   |   |-- landing/       # Landing page components
|   |   |   |-- layout/        # Navbar, backgrounds
|   |   |   |-- ui/            # Reusable styled components
|   |   |-- hooks/
|   |   |   |-- useWeatherChat.ts  # WebSocket chat hook
|   |   |   |-- useAutoScroll.ts
|   |   |   |-- useWeatherBackground.ts
|   |   |-- types/
|   |   |   |-- chat.ts        # Chat & WebSocket types
```

```
|   |   |   |-- weather.ts     # Weather data types
|   |   |-- config/
|   |       |-- constants.ts   # Intent labels, WS URL
|   |       |-- capabilities.ts
|   |       |-- weatherParticles.ts
|   |-- package.json
|   |-- vite.config.ts
|-- data/
|   |-- chroma_db/             # ChromaDB persistent storage
|   |-- docstore/              # Parent document JSON files
|   |-- pdfs/                  # Source PDF documents
|-- .claude/
|   |-- CLAUDE.md              # Project instructions for AI assistant
|   |-- plan.md                # Implementation plan with task tracking
|   |-- scope.md               # Project scope definition
|   |-- skills.md              # LangGraph patterns reference
|-- pyproject.toml             # Python project config (uv/hatchling)
|-- .env                       # Environment variables (not committed)
```

# 10 Environment Variables

| Variable | Required | Purpose |
|---|---|---|
| ANTHROPIC_API_KEY | Yes | Anthropic Claude API access |
| XWEATHER_CLIENT_ID | Yes | Xweather MCP server authentication |
| XWEATHER_CLIENT_SECRET | Yes | Xweather MCP server authentication |
| OPENAI_API_KEY | Yes (for RAG) | OpenAI embeddings for ChromaDB |
| LANGCHAIN_API_KEY | No | LangSmith tracing (auto-enables when set) |
| LANGCHAIN_PROJECT | No | LangSmith project name (default: "weatherwise") |
| LOG_LEVEL | No | Logging level (default: "INFO") |
| LOG_FORMAT | No | "console" (default) or "json" |
| CHECKPOINTER_MODE | No | "memory" (default), "sqlite", "redis", or "none" |
| REDIS_URL | Yes (if redis) | Redis connection string (default: "redis://localhost:6379") |
| CHECKPOINT_DB_PATH | No | SQLite DB path (default: "weather-wise_checkpoints.db") |
| SERVE_FRONTEND | No | Set to "1" to serve frontend static build |
| VITE_WS_URL | No (frontend) | WebSocket URL override for frontend |

# 11 Design Principles

1. **Reliability first.** No infinite loops. No race conditions. Degrade gracefully on failure. Every node has try/except with meaningful fallbacks.
2. **Two-phase execution.** Data-fetching tasks complete before calculation tasks. This is enforced structurally by the executor's routing logic, not by hope.
3. **Parallel-safe state.** Every state field that can be written by parallel workers uses an Annotated reducer. This is a hard rule — violating it causes silent data loss.
4. **RAG separation.** Heavy ingestion code (PDF parsing, image processing, LLM summarization) lives in `scripts/`. Runtime code in `src/` only does lightweight vector queries. Ingestion dependencies are never imported at runtime.
5. **Structured output everywhere.** The supervisor and planner use Pydantic models with `with_structured_output()` to guarantee parseable results from LLMs. No regex parsing of freeform text.
6. **Bounded execution.** The graph has a recursion limit of 25 steps. The executor has a max-iteration safeguard of 10. The calculator's tool loop is capped at 5 iterations. These bounds prevent runaway execution.
7. **Graceful degradation.** If the supervisor fails, it defaults to `real_time`. If the planner fails, it creates a single fetch task. If a fetcher fails, it returns an error string. If the synthesizer fails, it dumps the raw data. The system always produces some output.